



به نام خدا



آزمایشگاه سیستم عامل - بهار 1405

پروژه چهارم: همگام سازی در XV6

طراحان: امیرارسلان شهبازی - سیدمحمدجواد شریفی



مقدمه:

در این پروژه، مفاهیم اصلی همگام سازی در سیستم عامل XV6 به صورت عملی بررسی می شوند. هدف پروژه این است که دانشجویان اثر دسترسی همزمان به داده های مشترک را در هسته مشاهده کنند و با مشکلاتی مانند شرایط مسابقه، انتظار فعال، گرسنگی و کاهش کارایی آشنا شوند.

در بخش های مختلف پروژه، ابتدا تفاوت اجرای تک هسته ای و چند هسته ای با استفاده از شمارنده های فراخوانی سیستمی بررسی می شود. سپس مسئله ی تولیدکننده - مصرف کننده، قفل خواننده - نویسنده و قفل بلبلی پیاده سازی می شوند. این بخش ها به ترتیب نقش قفل ها، خواب و بیدارسازی، دسترسی اشتراکی و انحصاری، و عدالت در همگام سازی را نشان می دهند.

بخش اول: مشاهده اثر تکه‌هسته‌ای و چندهسته‌ای در همگام‌سازی

در این بخش، می‌خواهیم رفتار هسته‌ی XV6 را هنگام دسترسی همزمان به داده‌های مشترک بررسی کنیم. در سیستم‌های تکه‌هسته‌ای، چند پردازنده می‌توانند به صورت هم‌روند اجرا شوند، اما در هر لحظه تنها یک پردازنده کد هسته را اجرا می‌کند. در مقابل، در سیستم‌های چندهسته‌ای، چند پردازنده می‌توانند به صورت واقعی و همزمان وارد هسته شوند و به داده‌های مشترک دسترسی داشته باشند. این تفاوت باعث می‌شود راه‌حل‌های همگام‌سازی در این دو حالت همیشه یکسان نباشند.

برای مشاهده‌ی این مسئله، در این بخش یک ابزار ساده برای شمارش فراخوانی‌های سیستمی در هسته پیاده‌سازی می‌کنید. ابتدا اثر نبود همگام‌سازی را بررسی می‌کنید، سپس با استفاده از قفل چرخشی¹ درستی شمارنده را تضمین می‌کنید، و در نهایت با استفاده از داده‌های مختص هر پردازنده، راهکاری مقیاس‌پذیرتر برای سیستم‌های چندهسته‌ای پیاده‌سازی می‌کنید. هدف این بخش، درک عملی شرایط مسابقه²، ناحیه‌ی بحرانی³، رقابت روی قفل و مزیت استفاده از داده‌های مختص هر پردازنده است.

سوالات تئوری بخش اول

1. در هسته‌ی XV6، منظور از مسیر کنترلی چیست؟ تفاوت مسیرهای کنترلی ایجاد شده توسط فراخوانی سیستمی، وقفه و استثنا را توضیح دهید.
2. منظور از هسته‌ی با ورود مجدد⁴ چیست؟ سناریویی را توضیح دهید که در آن یک پردازنده در حال اجرای یک فراخوانی سیستمی است و در همان زمان یک وقفه رخ می‌دهد. چرا چنین وضعیتی می‌تواند باعث ایجاد شرایط مسابقه روی داده‌های مشترک هسته شود؟

¹ Spinlock

² Race condition

³ Critical section

⁴ Reentrant

3. تفاوت متن پردازش⁵ و متن وقفه⁶ را توضیح دهید. چرا کدی که در متن وقفه اجرا می‌شود نباید مسدود شود یا به خواب⁷ برود؟
4. عبارت زیر را در نظر بگیرید:

```
syscall_count[num]++;
```

توضیح دهید چرا این عبارت اتمیک نیست. یک سناریو با دو پردازنده ارائه کنید که در آن مقدار نهایی شمارنده به دلیل از دست رفتن به‌روزرسانی نادرست شود.

5. چرا در حالت تک‌هسته‌ای، نسخه‌ی بدون قفل شمارنده‌ی فراخوانی‌های سیستمی معمولاً مقدار صحیح تولید می‌کند، اما در حالت چندهسته‌ای ممکن است مقدار نهایی نادرست شود؟ تفاوت همروندی در تک‌هسته‌ای و اجرای موازی واقعی در چندهسته‌ای را توضیح دهید.
6. قفل چرخشی چگونه از ناحیه‌ی بحرانی محافظت می‌کند؟ چرا استفاده از قفل چرخشی برای شمارنده‌ی سراسری باعث درستی مقدار نهایی می‌شود، اما می‌تواند در حالت چندهسته‌ای باعث رقابت میان پردازنده‌ها شود؟
7. در xv6 هنگام گرفتن قفل چرخشی، وقفه‌ها روی پردازنده‌ی جاری غیرفعال می‌شوند. دلیل این کار چیست؟ سناریویی را توضیح دهید که در آن اگر وقفه‌ها غیرفعال نشوند، پردازنده می‌تواند دچار بن‌بست شود.
8. تفاوت وقفه‌های قابل ماسک و غیرقابل ماسک را توضیح دهید. منظور از غیرقابل ماسک بودن یک وقفه چیست و چرا سیستم‌عامل نمی‌تواند همه‌ی وقفه‌ها را به‌صورت دلخواه غیرفعال کند؟ در پاسخ خود توضیح دهید که چرا کدهای مربوط به وقفه‌های بحرانی باید کوتاه، سریع و بدون مسدود شدن اجرا شوند.
9. تفاوت استفاده‌ی مستقیم از دستورهای فعال و غیرفعال‌کردن وقفه مانند `cli/sti` با سازوکارهای تودرتو مانند `pushcli/popcli` یا معادل آن‌ها در xv6 چیست؟ چرا استفاده‌ی مستقیم از فعال‌کردن وقفه در نواحی بحرانی تودرتو می‌تواند خطرناک باشد؟

⁵ Process context

⁶ Interrupt context

⁷ Sleep

10. استفاده از شمارنده‌ی سراسری مشترک در چندهسته‌ای چه اثری روی کارایی و حافظه‌ی نهان دارد؟ توضیح دهید چرا استفاده از شمارنده‌های مختص هر پردازنده می‌تواند رقابت روی داده‌ی مشترک و سربار ناشی از نامعتبر شدن حافظه‌ی نهان را کاهش دهد.

سوالات عملی بخش اول

در این بخش می‌خواهیم بررسی کنیم که چرا راه‌حل‌های همگام‌سازی در سیستم تک‌هسته‌ای و چندهسته‌ای یکسان نیستند. برای این منظور، یک ابزار ساده برای شمارش فراخوانی‌های سیستمی در هسته xv6 پیاده‌سازی کنید و رفتار آن را در دو حالت `CPUS=1` و `CPUS=4` مقایسه نمایید.

در ابتدا یک شمارنده‌ی سراسری⁸ برای فراخوانی سیستمی تعریف کنید. هر بار که یک `syscall` اجرا می‌شود، شمارنده‌ی مربوط به آن `syscall` را افزایش دهید. سپس با اجرای یک برنامه‌ی سطح کاربر که چندین پردازش ایجاد می‌کند و هر پردازش تعداد زیادی `syscall` مانند `getpid` اجرا می‌کند، مقدار مورد انتظار و مقدار واقعی شمارنده را مقایسه کنید.

قسمت اول: شمارنده سراسری بدون قفل

ابتدا فراخوانی سیستمی شمارنده را بدون استفاده از هیچ قفل یا مکانیزم همگام‌سازی پیاده‌سازی کنید.

برای مثال:

```
syscall_count[num]++;
```

سپس یک `syscall` جدید با رابط زیر اضافه کنید:

```
int getcount(int syscall_number);
```

این `syscall` باید تعداد دفعات اجرای `syscall` مشخص‌شده را برگرداند.

⁸ Global

برنامه‌ی آزمایشی خود را یک بار در حالت تک‌هسته‌ای و یک بار در حالت چندهسته‌ای اجرا کنید. در گزارش خود مقدار مورد انتظار و مقدار واقعی را در هر دو حالت بنویسید. بررسی کنید که آیا در حالت چندهسته‌ای بخشی از به‌روزرسانی‌های شمارنده از دست می‌رود یا خیر. در گزارش خود توضیح دهید چرا عبارت `syscall_count[num]++` اتمیک نیست و چگونه اجرای همزمان آن روی چند پردازنده می‌تواند باعث شود مقدار نهایی شمارنده کمتر از مقدار مورد انتظار شود. اگر در برخی اجراها مقدار مشاهده‌شده با مقدار مورد انتظار برابر بود، توضیح دهید چرا این موضوع به معنای ایمن بودن پیاده‌سازی بدون قفل نیست.

قسمت دوم: ایمن‌سازی شمارنده‌ی سراسری با قفل چرخشی

در قسمت قبل مشاهده کردید که افزایش مستقیم یک شمارنده‌ی سراسری در حالت چندهسته‌ای می‌تواند باعث از دست رفتن برخی به‌روزرسانی‌ها شود. در این قسمت، پیاده‌سازی قبلی خود را به‌گونه‌ای تغییر دهید که دسترسی به شمارنده‌ی سراسری به‌صورت همگام‌سازی‌شده انجام شود.

برای این کار، یک قفل چرخشی در هسته تعریف کنید و از آن برای محافظت از ناحیه‌ای استفاده کنید که در آن مقدار شمارنده‌ی فراخوانی سیستمی افزایش داده می‌شود. به عبارت دیگر، هر پردازنده پیش از تغییر مقدار شمارنده باید قفل را بگیرد و پس از پایان تغییر مقدار، قفل را آزاد کند.

ساختار کلی پیاده‌سازی می‌تواند به شکل زیر باشد:

```
struct spinlock syscall_count_lock;
```

برنامه‌ی آزمایشی قسمت قبل را دوباره در دو حالت تک‌هسته‌ای و چندهسته‌ای اجرا کنید.

در گزارش خود، مقدار مورد انتظار و مقدار مشاهده‌شده را در هر دو حالت نشان دهید و بررسی کنید که آیا مشکل از دست رفتن به‌روزرسانی‌ها برطرف شده است یا خیر. همچنین توضیح دهید چرا استفاده از قفل چرخشی باعث درستی مقدار نهایی می‌شود، اما در حالت چندهسته‌ای می‌تواند باعث ایجاد رقابت میان پردازنده‌ها برای گرفتن قفل شود و کارایی کاهش می‌یابد.

قسمت سوم: بهینه‌سازی شمارنده‌ها با استفاده از داده‌های مختص هر پردازنده

در قسمت قبل، با استفاده از قفل چرخشی توانستید از درستی مقدار شمارنده‌ی سراسری محافظت کنید. با این حال، در یک سیستم چند هسته‌ای، استفاده از یک شمارنده‌ی مشترک باعث می‌شود همه‌ی پردازنده‌ها برای تغییر یک داده‌ی واحد با یکدیگر رقابت کنند. این موضوع می‌تواند باعث کاهش کارایی و افزایش سربار همگام‌سازی شود.

در این قسمت، پیاده‌سازی خود را به گونه‌ای تغییر دهید که به جای یک شمارنده‌ی سراسری مشترک، برای هر پردازنده یک مجموعه شمارنده‌ی جداگانه داشته باشید. در این روش، هر پردازنده فقط شمارنده‌ی مربوط به خودش را به‌روزرسانی می‌کند و مقدار نهایی در زمان گزارش‌گیری از مجموع شمارنده‌های همه‌ی پردازنده‌ها به دست می‌آید.

ساختار کلی پیاده‌سازی می‌تواند به شکل زیر باشد:

```
uint64 syscall_count_cpu[NCPU][NSYSCALL];
```

هنگام اجرای هر فراخوانی سیستمی، ابتدا شناسه‌ی پردازنده‌ی جاری را به دست آورید و سپس فقط شمارنده‌ی مربوط به همان پردازنده را افزایش دهید. برای به دست آوردن شناسه‌ی پردازنده‌ی جاری، دقت کنید که شرایط لازم برای استفاده‌ی صحیح از تابع مربوط به شناسه‌ی پردازنده در xv6 رعایت شود.

در ادامه، فراخوانی‌های سیستمی زیر را پیاده‌سازی کنید:

```
int getcount(int syscall_number);
```

```
int getcpucount(int cpu_id, int syscall_number);
```

فراخوانی سیستمی `getcount` باید مجموع تعداد اجرای فراخوانی سیستمی موردنظر را روی همه‌ی پردازنده‌ها برگرداند. فراخوانی سیستمی `getcpucount` باید تعداد اجرای همان فراخوانی سیستمی را فقط برای پردازنده‌ی مشخص شده برگرداند.

برنامه‌ی آزمایشی خود را به‌گونه‌ای تغییر دهید که پس از اجرای بار کاری، علاوه بر مقدار کل، مقدار شمارنده‌ی هر پردازنده را نیز به‌صورت جداگانه چاپ کند. نمونه‌ی خروجی می‌تواند به شکل زیر باشد:

CPU 0 getpid count: 201244

CPU 1 getpid count: 198101

CPU 2 getpid count: 203450

CPU 3 getpid count: 197205

Total getpid count: 800000

Expected count: 800000

در گزارش خود، خروجی نسخه‌ی سراسری همراه با قفل چرخشی را با نسخه‌ی مختص هر پردازنده مقایسه کنید. توضیح دهید چرا در نسخه‌ی مختص هر پردازنده، رقابت روی داده‌ی مشترک کاهش می‌یابد و چرا این روش برای سیستم‌های چند هسته‌ای مقیاس‌پذیرتر است.

بخش دوم: پیاده‌سازی تولیدکننده - مصرف‌کننده در هسته

xv6

در این بخش، یکی از مسائل کلاسیک همگام‌سازی، یعنی مسئله‌ی تولیدکننده - مصرف‌کننده، در محیط xv6 بررسی می‌شود. هدف اصلی این بخش، درک تفاوت میان محافظت از داده‌ی مشترک و منتظر ماندن برای تغییر وضعیت یک منبع مشترک است. در چنین مسئله‌ای، تنها جلوگیری از دسترسی همزمان نادرست کافی نیست؛ بلکه باید مشخص شود پردازنده‌ها هنگام آماده نبودن شرایط، چگونه بدون مصرف بیهوده‌ی پردازنده منتظر بمانند.

سوالات تئوری بخش دوم

1. مسئله‌ی تولیدکننده - مصرف‌کننده را توضیح دهید. چرا وجود یک بافر محدود بین تولیدکننده و مصرف‌کننده می‌تواند باعث نیاز به همگام‌سازی شود؟

2. تفاوت قفل چرخشی و خوابیدن پردازنده را توضیح دهید. چرا قفل چرخشی برای محافظت از یک ناحیه‌ی بحرانی کوتاه مناسب است، اما برای انتظار طولانی‌مدت روی پر یا خالی شدن بافر مناسب نیست؟
3. انتظار فعال⁹ چیست و چه اثری روی مصرف پردازنده دارد؟ چرا برنامه‌ای که به‌صورت مکرر یک شرط را بررسی می‌کند، حتی بدون انجام کار مفید می‌تواند پردازنده را مصرف کند؟
4. در مسئله‌ی تولیدکننده - مصرف‌کننده، چرا تولیدکننده نباید هنگام پر بودن بافر به‌صورت فعال منتظر بماند و مصرف‌کننده نباید هنگام خالی بودن بافر به‌صورت فعال منتظر بماند؟

سوالات عملی بخش دوم

در این بخش، مسئله‌ی تولیدکننده - مصرف‌کننده را در هسته xv6 پیاده‌سازی می‌کنید. برای این منظور، یک بافر محدود در هسته تعریف می‌شود. پردازنده‌های سطح کاربر می‌توانند نقش تولیدکننده یا مصرف‌کننده داشته باشند. تولیدکننده‌ها با استفاده از یک فراخوانی سیستمی، مقدارهایی را داخل بافر قرار می‌دهند و مصرف‌کننده‌ها با استفاده از فراخوانی سیستمی دیگر، مقدارها را از بافر خارج می‌کنند.

در این طراحی، بافر داخل هسته یک داده‌ی مشترک است. بنابراین چند پردازنده ممکن است همزمان بخواهند به متغیرهای داخلی بافر دسترسی داشته باشند. هدف این بخش آن است که ابتدا از بافر مشترک با قفل چرخشی محافظت کنید و سپس پیاده‌سازی را به‌گونه‌ای تغییر دهید که تولیدکننده‌ها و مصرف‌کننده‌ها هنگام پر یا خالی بودن بافر، به جای انتظار فعال و مصرف بیهوده‌ی پردازنده، به خواب بروند و در زمان مناسب بیدار شوند.

برای ساده‌سازی، ساختار بافر، متغیرهای مربوط به آن، و پیاده‌سازی فراخوانی‌های سیستمی `produce` و `consume` را در فایل `sysproc.c` قرار دهید. نیازی به ایجاد فایل جدید در هسته نیست. قفل مربوط به بافر باید هنگام راه‌اندازی هسته مقداردهی اولیه شود.

در این بخش دو فراخوانی سیستمی زیر را پیاده‌سازی کنید:

```
int produce(int value);
```

⁹ Busy wait


```
int consume(void);
```

فراخوانی سیستمی produce باید مقدار ورودی را در بافر هسته قرار دهد. فراخوانی سیستمی consume باید یک مقدار را از بافر خارج کرده و آن را به برنامه‌ی سطح کاربر برگرداند.

قسمت اول: پیاده‌سازی تولیدکننده - مصرف‌کننده با قفل چرخشی

در این قسمت، یک بافر محدود در هسته تعریف کنید که بتواند تعدادی عدد صحیح را نگهداری کند. برای مدیریت بافر، از یک آرایه، اندیس شروع، اندیس پایان و یک شمارنده برای تعداد عناصر موجود در بافر استفاده کنید. ساختار کلی بافر می‌تواند شامل موارد زیر باشد:

```
int buffer[BUFFER_SIZE];
```

```
int head;
```

```
int tail;
```

```
int count;
```

```
struct spinlock lock;
```

دسترسی به تمام بخش‌های مشترک بافر باید با قفل چرخشی محافظت شود. بنابراین هر بار که تولیدکننده یا مصرف‌کننده می‌خواهد به head، tail، buffer یا count دسترسی داشته باشد، ابتدا باید قفل را بگیرد و پس از پایان کار، قفل را آزاد کند.

در این قسمت، رفتار فراخوانی‌ها غیر مسدود شونده باشد. یعنی اگر تولیدکننده بخواند مقداری را وارد بافر کند اما بافر پر باشد، فراخوانی سیستمی produce مقدار 1- برگرداند. همچنین اگر مصرف‌کننده بخواند مقداری را از بافر خارج کند اما بافر خالی باشد، فراخوانی سیستمی consume مقدار 1- برگرداند.

برنامه‌ی آزمایشی در سطح کاربر بنویسید که چند پردازشی تولیدکننده و چند پردازشی مصرف‌کننده ایجاد کند. هر تولیدکننده باید یک بازه‌ی مشخص از اعداد را تولید کند تا خروجی قابل بررسی باشد. برای مثال، تولیدکننده‌ی اول اعداد 100 تا 150، تولیدکننده‌ی دوم اعداد 200 تا 250 و تولیدکننده‌ی سوم اعداد 300 تا 350 را تولید کند.

مصرف‌کننده‌ها باید مقدارهای دریافت‌شده را چاپ کنند. نمونه‌ی خروجی می‌تواند به شکل زیر باشد:

```
producer 1 produced 1000
producer 2 produced 2000
consumer 1 consumed 1000
consumer 2 consumed 2000
```

در گزارش خود توضیح دهید که چرا head، tail، buffer و count داده‌های مشترک محسوب می‌شوند و چرا دسترسی به آن‌ها باید با قفل چرخشی محافظت شود. همچنین توضیح دهید چرا این نسخه با وجود استفاده از قفل، هنوز از نظر انتظار مناسب نیست؛ زیرا در صورت پر بودن یا خالی بودن بافر، پردازنده‌ها مجبور می‌شوند دوباره تلاش کنند.

قسمت دوم: حذف انتظار فعال با استفاده از خواب و بیدارسازی¹⁰

در قسمت قبل، اگر بافر پر بود، produce مقدار 1- برمی‌گرداند و اگر بافر خالی بود، consume مقدار 1- برمی‌گرداند. در نتیجه، برنامه‌ی سطح کاربر ممکن است برای موفق‌شدن عملیات، بارها و بارها produce یا consume را صدا بزند. این روش نوعی انتظار فعال است و باعث مصرف بی‌هوده‌ی پردازنده می‌شود.

در این قسمت، پیاده‌سازی را به‌گونه‌ای تغییر دهید که تولیدکننده و مصرف‌کننده به جای برگشت خطا، در صورت نیاز منتظر بمانند.

رفتار مورد انتظار در نسخه‌ی جدید به این صورت است:

اگر تولیدکننده بخواهد مقداری را وارد بافر کند اما بافر پر باشد، تولیدکننده باید به خواب برود تا زمانی که یک مصرف‌کننده مقداری را از بافر خارج کند و فضای خالی ایجاد شود.

اگر مصرف‌کننده بخواهد مقداری را از بافر خارج کند اما بافر خالی باشد، مصرف‌کننده باید به خواب برود تا زمانی که یک تولیدکننده مقدار جدیدی وارد بافر کند.

¹⁰ Wakeup

پس از هر تولید موفق، مصرف‌کننده‌های منتظر باید بیدار شوند.
پس از هر مصرف موفق، تولیدکننده‌های منتظر باید بیدار شوند.

برای این قسمت از سازوکار sleep و wakeup در xv6 استفاده کنید. دقت کنید که بررسی پر یا خالی بودن بافر و به خواب رفتن پردازنده باید به‌درستی و همراه با قفل انجام شود تا بیدارسازی از دست نرود.

در نسخه‌ی مسدود شونده، produce و consume نباید در حالت پر یا خالی بودن بافر با خطا بازگردند. اگر انجام عملیات در آن لحظه ممکن نباشد، پردازنده‌ی فراخواننده باید با استفاده از سازوکار sleep به خواب برود و پس از تغییر وضعیت بافر توسط پردازنده‌ی مقابل، با wakeup بیدار شود و عملیات خود را ادامه دهد.

برنامه‌ی آزمایشی قسمت قبل را دوباره اجرا کنید. این بار تولیدکننده‌ها و مصرف‌کننده‌ها نباید با بررسی مداوم بافر منتظر بمانند. خروجی برنامه مانند بخش قبل باید نشان دهد که مقدارهای تولید شده در نهایت توسط مصرف‌کننده‌ها دریافت می‌شوند.

در گزارش خود، نسخه‌ی غیرمسدودشونده‌ی قسمت اول را با نسخه‌ی مسدودشونده‌ی قسمت دوم مقایسه کنید. توضیح دهید چرا قفل چرخشی برای محافظت از خود بافر لازم است، اما برای انتظار طولانی‌مدت کافی نیست. همچنین توضیح دهید چرا sleep و wakeup باعث می‌شوند پردازنده‌های منتظر بدون هدر دادن پردازنده منتظر بمانند.

بخش سوم: پیاده‌سازی قفل خواننده-نویسنده و حل مشکل گرسنگی نویسنده

در این بخش، یکی دیگر از مسائل کلاسیک همگام‌سازی، یعنی مسئله‌ی خوانندگان و نویسندگان (Readers-Writers) در محیط xv6 بررسی می‌شود. در بسیاری از سیستم‌عامل‌ها، برخی از ساختارهای داده بیشتر خوانده می‌شوند و کمتر تغییر می‌کنند. استفاده از قفل‌های انحصاری معمولی (مانند قفل چرخشی¹¹ یا خوابنده¹²) برای چنین داده‌هایی باعث کاهش کارایی می‌شود. زیرا در صورتی که هیچ

¹¹ SpinLock

¹² SleepLock

پردازه‌ای در حال نوشتن نباشد، چند پدازه می‌توانند به صورت همزمان بخوانند بدون این که مشکلی به وجود بیاید.

هدف اصلی این بخش، درک تفاوت میان دسترسی اشتراکی و انحصاری به داده‌های مشترک و چالش مدیریت اولویت‌هاست. در این مسئله بررسی می‌شود که چگونه می‌توان با کنترل هوشمندانه‌ی ورود به ناحیه‌ی بحرانی، از بروز مشکل گرسنگی¹³ جلوگیری کرد و تخصیص منابع را در شرایط رقابت نابرابر مدیریت نمود.

سوالات تئوری بخش سوم

1. مسئله‌ی خوانندگان و نویسندگان را توضیح دهید. چرا استفاده از قفل انحصاری معمولی برای منبعی که دفعات خواندن آن بسیار بیشتر از نوشتن است، کارایی سیستم را کاهش می‌دهد؟
2. در یک قفل خواننده-نویسنده ساده، اگر همواره خواننده‌های جدیدی از راه برسند و قفل خواندن را درخواست کنند، چه مشکلی برای پدازه‌ی نویسنده به وجود می‌آید؟ با یک سناریو توضیح دهید.
3. برای حل مشکل گرسنگی نویسنده، رویکرد حق تقدم با نویسنده (Writer Priority) مطرح می‌شود. این رویکرد چگونه کار می‌کند؟ آیا با اعمال این سیاست، امکان بروز گرسنگی برای خوانندگان وجود دارد؟ چرا؟
4. در پیاده‌سازی قفل خواننده-نویسنده، چرا نیاز داریم از یک قفل چرخشی داخلی برای محافظت از متغیرهای وضعیت قفل (مانند تعداد خوانندگان) استفاده کنیم؟

سوالات عملی بخش سوم

در این بخش، قفل خواننده-نویسنده را در هسته xv6 پیاده‌سازی می‌کنید. این قفل به چندین پدازه اجازه می‌دهد تا همزمان برای خواندن وارد ناحیه بحرانی شوند. اما اگر پدازه‌ای قصد نوشتن داشت، باید دسترسی کاملاً انحصاری داشته باشد؛ یعنی فقط همان پدازه می‌تواند قفل را بگیرد و وارد ناحیه بحرانی شود. همچنین، قفل شما باید از گرسنگی نویسندگان جلوگیری کند؛ به این معنی که اگر یک

¹³ Starvation

نویسنده منتظر ورود به ناحیه بحرانی است، خوانندگان جدیدی که پس از نویسنده درخواست قفل را می‌دهند نباید اجازه ورود پیدا کنند (حتی اگر قفل دست خواننده‌ها باشد) و باید منتظر بمانند تا در ابتدا نویسنده قفل را در اختیار بگیرد.

قسمت اول: تعریف ساختار قفل و توابع مورد نیاز

ابتدا ساختار داده‌ی مربوط به قفل خواننده-نویسنده را در هسته تعریف کنید. این ساختار باید بتواند وضعیت فعلی قفل را مدیریت کند. ساختار کلی قفل می‌تواند شامل موارد زیر باشد:

- `int readers`: تعداد خوانندگان فعلی.
- `int writer_active`: آیا نویسنده‌ای در حال نوشتن هست یا نه.
- `int writer_waiting`: تعداد نویسندگانی که منتظر ورود هستند.
- `struct spinlock lock`: برای محافظت از دسترسی به متغیرهای قفل.

سپس توابع مربوط به مقداردهی اولیه و چهار عملیات اصلی این قفل را در هسته پیاده‌سازی کنید:

- `void rwlock_init(struct rwlock *rw)`: مقداردهی اولیه قفل.
- `void rwlock_acquire_read(struct rwlock *rw)`: درخواست قفل برای خواندن.
- `void rwlock_release_read(struct rwlock *rw)`: آزادسازی قفل خواندن.
- `void rwlock_acquire_write(struct rwlock *rw)`: درخواست قفل برای نوشتن.
- `void rwlock_release_write(struct rwlock *rw)`: آزادسازی قفل نوشتن.

توجه کنید که پیاده‌سازی را باید طوری انجام دهید که مشکل گرسنگی نویسنده ایجاد نشود.

قسمت دوم: ارزیابی در فضای کاربر

برای اثبات صحت عملکرد قفل خود، فراخوانی‌های سیستمی مناسبی بنویسید تا بتوانید توابع قفل را از برنامه‌های سطح کاربر صدا بزنید.

سپس یک برنامه‌ی آزمایشی سطح کاربر بنویسید که چندین پردازش خواننده و حداقل یک پردازش نویسنده ایجاد کند.

سناریوی برنامه باید به گونه‌ای باشد که:

1. ابتدا چند خواننده وارد ناحیه بحرانی شوند و یک انتظار نسبتاً طولانی داشته باشند.
2. در همین حین، یک نویسنده درخواست ورود بدهد.
3. بلافاصله پس از درخواست نویسنده، چند خواننده‌ی جدید نیز درخواست ورود بدهند.

در گزارش خود، خروجی این برنامه را تحلیل کنید. نشان دهید خوانندگان جدیدی که پس از نویسنده درخواست داده‌اند، با وجود اینکه خوانندگان اولیه هنوز در ناحیه بحرانی حضور دارند، به داخل ناحیه بحرانی راه پیدا نمی‌کنند و منتظر می‌مانند تا نویسنده کار خود را انجام دهد. این خروجی اثبات می‌کند که پیاده‌سازی شما مشکل گرسنگی نویسنده را با موفقیت حل کرده است.

بخش چهارم: پیاده‌سازی قفل بلیطی¹⁴ و برقراری عدالت¹⁵

در این بخش، راهکار دیگری برای همگام‌سازی تحت عنوان قفل بلیطی (Ticket Lock) بررسی می‌شود. در قفل‌های استاندارد xv6 مانند قفل چرخشی، سیاست اعطای قفل بر اساس شانس و رقابت سخت‌افزاری است که در شرایط رقابت شدید می‌تواند منجر به گرسنگی پردازنده‌ها شود. برای حل این مشکل، قفل بلیطی با ایجاد یک سازوکار نوبت‌دهی قانون‌مند و مبتنی بر صف، منطق ورود به ناحیه بحرانی را مدیریت می‌کند.

هدف اصلی این بخش، درک عملی مفهوم عدالت در همگام‌سازی، جبران ضعف رقابت‌های سخت‌افزاری ناعادلانه و آشنایی با نحوه ساختاردهی به صف‌های انتظار متقاضیان قفل است.

سوالات تئوری بخش چهارم

1. نحوه کارکرد قفل بلیطی را به صورت مختصر توضیح دهید.
2. تفاوت اصلی قفل چرخشی استاندارد در xv6 با قفل بلیطی از منظر عدالت چیست؟
3. قفل اولویت‌دار (Priority Lock) را با قفل بلیطی (Ticket Lock) از سه منظر زیر مقایسه کنید:

¹⁴ Ticket Lock

¹⁵ Fairness

- عدالت: کدامیک عادلانه‌تر رفتار می‌کند؟
 - پیچیدگی: پیاده‌سازی کدامیک سر بار محاسباتی کمتری دارد؟
 - احتمال گرسنگی: کدامیک در برابر گرسنگی ایمن است؟
4. با وجود اینکه قفل بلیطی کاملاً عادلانه است، اما در سیستم‌های چند هسته‌ای (Multi-core) با تعداد هسته بالا همچنان دارای مشکل عملکردی در خصوص ترافیک حافظه نهان (Cache Coherency) است. با استفاده از پروتکل‌های همگامی کش (مانند MESI) دلیل این افت کارایی را توضیح دهید.
5. در ساختار داخلی قفل بلیطی، یک قفل چرخشی وجود دارد. هدف از وجود این قفل داخلی چیست؟ توضیح دهید اگر دو پردازنده بخواهند همزمان و بدون استفاده از این قفل، مقدار بلیط را برای خود بردارند و افزایش دهند، چه مشکلی رخ می‌دهد؟ (به شرایط مسابقه¹⁶ اشاره کنید).

سوالات عملی بخش چهارم (امتیازی)

در این بخش، باید قفل بلیطی را در سطح هسته‌ی xv6 پیاده‌سازی کنید. منطق کلی این قفل بر پایه‌ی دو شمارنده‌ی اصلی بنا شده است: یکی برای توزیع شماره بلیط بین پردازنده‌های متقاضی و دیگری برای نشان دادن نوبت فعلی سیستم.

هر پردازنده‌ای که قصد ورود به ناحیه بحرانی را دارد، ابتدا باید به صورت ایمن یک شماره بلیط اختصاصی برای خود دریافت کرده و شمارنده‌ی توزیع بلیط را برای نفرات بعدی یک واحد افزایش دهد. سپس پردازنده در یک حلقه‌ی انتظار قرار می‌گیرد و تنها زمانی می‌تواند وارد ناحیه بحرانی شود که نوبت فعلی سیستم با شماره بلیط او برابر شود. پس از اتمام کار و در زمان خروج از ناحیه بحرانی، پردازنده مالک قفل موظف است متغیر نوبت سیستم را یک واحد افزایش دهد تا پردازنده منتظر بعدی بتواند وارد شود. این سازوکار، رعایت دقیق ترتیب ورود (FIFO) را تضمین می‌کند.

¹⁶ Race Condition

قسمت اول: تعریف ساختار قفل و توابع مورد نیاز

ابتدا ساختار داده‌ی مربوط به قفل بلیطی را در هسته تعریف کنید. ساختار کلی این قفل می‌تواند شامل موارد زیر باشد:

- `uint ticket`: شماره بلیط بعدی که قرار است به متقاضی داده شود.
- `uint turn`: شماره نوبت فعلی که اجازه ورود به ناحیه بحرانی را دارد.
- `struct spinlock lock`: برای محافظت از دسترسی به متغیرهای قفل (همگام سازی عملیات برداشتن بلیط)

سپس توابع زیر را برای مدیریت این قفل در هسته پیاده‌سازی کنید:

- `void ticketLock_init(struct ticketLock *tl)`: مقدار دهی اولیه قفل.
- `void ticketLock_acquire(struct ticketLock *tl)`: درخواست قفل.
- `void ticketLock_release(struct ticketLock *tl)`: آزادسازی قفل.

قسمت دوم: ارزیابی در فضای کاربر

برای تست عملکرد و اثبات عادلانه بودن این قفل، فراخوانی‌های سیستمی مناسبی طراحی کنید تا برنامه‌های سطح کاربر بتوانند این قفل را دریافت و آزاد کنند.

سپس یک برنامه‌ی آزمایشی سطح کاربر بنویسید که حداقل ۵ پردازشی فرزند ایجاد کند. این پردازها باید با فاصله‌ی زمانی کوتاه درخواست ورود به ناحیه بحرانی را ارسال کنند.

در سناریوی شما:

1. پردازها هنگام درخواست قفل باید پیامی چاپ کنند. این پیام باید شامل شماره شناسه (PID) و شماره بلیطشان باشد. (راهنمایی: از آنجا که تخصیص بلیط در داخل هسته انجام می‌شود، می‌توانید این پیام را با استفاده از تابع `cprintf` در زمان دریافت بلیط، درون خود هسته چاپ کنید).

2. ناحیه بحرانی باید شامل یک تاخیر زمانی (مثلاً یک حلقه محاسباتی طولانی) باشد تا رقابت روی قفل شکل بگیرد.

3. پردازنده‌ها پس از ورود به ناحیه بحرانی نیز باید پیامی چاپ کنند. این پیام باید شامل شماره شناسه، شماره بلیط و شماره نوبت باشد.

در گزارش خود خروجی این برنامه را تحلیل کنید. خروجی شما باید به وضوح نشان دهد پردازنده‌ها دقیقاً به همان ترتیبی که قفل را درخواست کرده‌اند (به صورت FIFO) وارد ناحیه بحرانی می‌شوند و هیچ پردازنده‌ای در صف جا نمی‌ماند.

نکات تحویل

- پروژه خود را در یک مخزن خصوصی در Github یا Gitlab پیش برده و در نهایت یک نفر از اعضای گروه کدها را به همراه پوشه git. و فایل pdf گزارش کار زیپ کرده و در سامانه با فرمت OS-Lab4-<SID1>-<SID2>-<SID3>.zip آپلود نمایید.
- رعایت نکردن مورد فوق کسر نمره را به همراه خواهد داشت.
- به تمامی سوالات پاسخ داده و پاسخ‌ها و نتایج را در گزارش کار خود قید کنید.
- بخش خوبی از نمره شما را پاسخ دهی به سوالات مطرح شده تشکیل می‌دهد که به شما در درک نحوه کارکرد xv6 کمک میکند.
- در پیاده‌سازی خود در خصوص مواردی که ذکر نشده‌اند می‌توانید فرضی منطقی و بر اساس کارکرد shell لینوکس خود در نظر بگیرید.
- تمامی مواردی که در جلسه توجیهی، و فروم درس مطرح می‌شوند، جزئی از پروژه خواهند بود. در صورت وجود هرگونه سوال یا ابهام می‌توانید با ایمیل دستیاران مربوطه یا گروه تیمز درس در ارتباط باشید.
- همه اعضای گروه باید به تمام بخش‌های پروژه آپلود شده توسط گروه، چه کد و چه سوالات مسلط بوده و هنگام تحویل از تمامی اعضا و از تمامی قسمت‌ها به صورت تصادفی سوال پرسیده خواهد شد.
- استفاده از مدل‌های زبانی تنها به عنوان دستیار مجاز است. در صورت مشاهده هرگونه شباهت بین کدها یا گزارش‌ها میان گروه‌ها در این ترم یا ترم‌های گذشته، مطابق با سیاست‌های درس برخورد خواهد شد.

موفق باشید